TP 03 :
Advanced Encryption Standard (AES)

Master 1 CYBERSECURITY ET INTELLIGENCE ARTIFICIAL

ENS : KADDACHE M.E.A **2025-2026**

Before the start :

```
pip install pycryptodome
```

Part 1 : First Encryption

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
from Crypto.Random import get_random_bytes

key = get_random_bytes(16)   # 16 bytes = 128 bit
cipher = AES.new(key, AES.MODE_ECB) # ECB Mode

data = b"AES with M1 Cysia Students"
ciphertext = cipher.encrypt(pad(data, 16))

print("Ciphertext:", ciphertext)

# Decryption
decipher = AES.new(key, AES.MODE_ECB)
plaintext = unpad(decipher.decrypt(ciphertext), 16)

print("Recovered:", plaintext)
```

1. Why do we use **pad() ?**
2. What happens if we remove **pad() ?**

Try this

Change Data and run code :

```
data = b"MMMMMMMMMMMMMMMMAAAAAAAAAAAAAAAAMMMMMMMMMMMMMMMMAAAAAAAAAAAAAAAA"
```

1. **What do you notice in the encryption and why?**

Change Data and Key and run code :

```
key = b"KADDACHECYSIAM01"
data = b""
print(pad(data, 16)) #add this line after data = b""
```

1. **What do you notice in the encryption and why?**

Part 2 : Understanding Modes

CBC :

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
from Crypto.Random import get_random_bytes

key = get_random_bytes(16)
iv = get_random_bytes(16)

cipher = AES.new(key, AES.MODE_CBC, iv)
data = b"Attack at dawn!!!"
ciphertext = cipher.encrypt(pad(data, 16))
print("IV:", iv)
print("Ciphertext:", ciphertext)
# Decrypt
decipher = AES.new(key, AES.MODE_CBC, iv)
plaintext = unpad(decipher.decrypt(ciphertext), 16)
print("Recovered:", plaintext)
```

1) **Change one byte in the ciphertext before decrypting.What happens?**
   a) **Explain mathematically its effect on:**
      i) **The current block**
      ii) **The next block**
2) **Change IV and key and run code :**

```
key = b"KADDACHE AMINE 1"
iv = b"KADDACHE AMINE 1"
```

   a) **What do you notice in the encryption ?**

CTR :

```
from Crypto.Cipher import AES

key = b"KADDACHE AMINE 1"
non=b"KADDACHE"
cipher = AES.new(key, AES.MODE_CTR,nonce=non)
data = b"Modern cryptography is beautiful"
ciphertext = cipher.encrypt(data)
print("Ciphertext:", ciphertext)
```

```python
# decrypt
decipher = AES.new(key, AES.MODE_CTR, nonce=non)
plaintext = decipher.decrypt(ciphertext)
print("Recovered:", plaintext)
```

1. **Why we don't need a pad() in CTR ?**
2. **If one block of the encrypted message is corrupted, how many blocks can we recover?**

Part 3 : File encryption

PDF :

```python
from Crypto.Cipher import AES

# read PDF
with open("input.pdf", "rb") as f:
    data = f.read()

# Key و nonce
key = b"KKKKKKKKKKKKKKKK"   # 16 bytes = AES-128
nonce = b"AAAABBBB"         # 8 bytes (مناسب لـ CTR)

cipher = AES.new(key, AES.MODE_CTR, nonce=nonce)
ciphertext = cipher.encrypt(data)

with open("encrypted.bin", "wb") as f:
    f.write(ciphertext)

print("Encrypted size:", len(ciphertext))
print("First 64 bytes (hex):", ciphertext[:64].hex())
```

1. **Run the code and observe the start of encryption. The first 4 bytes in PDF should start with what?**
2. **Add decryption code**
3. **Save data after decrypting it in PDF**
4. **Ensure file integrity after decryption**
5. **Now try re-encrypting and save only the encrypted file. Replace .bin with .pdf and open the resulting file. What do you notice?**